CourseForge

Interview Preparation Pack

AI-powered course generator — full-stack project deep-dive

Live demo: text2course-nu.vercel.app
API: text-to-course.onrender.com

CourseForge — Interview Prep

A deep-dive into the architecture, stack, design choices, and likely interview questions for the CourseForge project (AI-powered course generator).

1. Project Overview

CourseForge accepts a free-text topic and generates a complete structured course: 3–6 modules with 3–5 lessons each, where each lesson contains explanatory paragraphs, code examples, an embedded YouTubereference, and 5–8 interactive MCQs. Users authenticate via Auth0, courses are persisted in MongoDB, and individual lessons can be exported as searchable PDFs for offline study.

The architecture optimizes for three things: (1) reliable JSON output from non-deterministic LLMs, (2) low LLM cost via aggressive caching + provider fallback, and (3) deploy-friendliness on free-tier cloud services.

Live URLs: frontend at https://text2course-nu.vercel.app, API at https://text-to-course.onrender.com.

2. Tech Stack — Detailed

Frontend

React 19 + Vite — Vite was chosen over Create React App or Next.js because the app is a pure SPA with no server-side rendering needs. Vite gives sub-200ms hot reload during development, ESM-first builds, and produces ~900KB minified bundles. React 19 is used for `use`-style data loading patterns and improved concurrent rendering.

React Router v7 — Client-side routing with nested guards (`ProtectedRoute`, `GuestRoute`) that read Auth0 state to gate access. A `vercel.json` rewrite ensures direct hits to `/dashboard`, `/courses/:id` etc. fall back to `index.html` so the SPA can route internally.

Auth0 React SDK (`@auth0/auth0-react`) — Wraps the app in `<Auth0Provider>`, exposes `loginWithRedirect`, `logout`, `getAccessTokenSilently`. Tokens are cached in `localStorage` with refresh tokens enabled, so sessions survive across days.

jsPDF — Used to render lessons as paginated PDFs directly via text APIs (no html2canvas). Output is searchable, selectable, small (~30 KB per lesson), and renders cleanly at page seams.

lucide-react — SVG icon library, tree-shakable, ~1 KB per icon.

react-hot-toast — Lightweight notification library, themed via CSS variables.

Plain CSS with design tokens — No Tailwind. The app uses a custom CSS-variable system in `index.css` for typography (Inter + Merriweather + JetBrains Mono), spacing, radii, colors, shadows, and animations. Light and dark themes are switched by toggling `data-theme` on `<html>`.

Backend

Node.js + Express — Standard. Express keeps the surface area small (routes + middleware), and Node's `fetch` is used for outbound calls (Groq, Gemini, YouTube)so we avoid axios on the server.

Mongoose — ODM for MongoDB. Schemas are defined for `Course`, `Module`, `Lesson` with explicit refs forming a Course !' Module !' Lesson hierarchy.

Zod — Runtime schema validation for AI output. Distinguishes "JSON parsed" from "JSON is semantically correct" — catches cases where the LLM returns valid JSON with wrong shapes (e.g., 2 modules instead of 3–6).

express-oauth2-jwt-bearer — Auth0's official middleware. Verifies RS256 JWTs against the configured issuer and audience. Sets `req.auth.payload`; we lift `sub` into `req.user.id` and use that as the canonical user identifier on Course documents.

redis (node-redis v5) — Connects to Upstash Redis over TLS. Used for caching AI outputs and request coalescing.

LangChain (classic + google-genai packages) — Only loaded for optional RAG (Retrieval-Augmented Generation). Gated behind `RAG_ENABLED=false` by default to avoid embedding tokens cost on Gemini's free tier.

External Services

MongoDB Atlas (M0 free tier) — Cloud-hosted MongoDB. 512MB storage, shared cluster. Connection string drives `MONGO_URI`.

Upstash Redis — Cloud-hosted Redis with TLS. 10K commands/day on free tier. URL format is `rediss://default:token@host:6379` (note `rediss://` for TLS).

Auth0 — OIDC identity provider. We use the Single Page Application type with Authorization Code + PKCE flow. The custom API audience (`https://text-to-course-api`) is what makes Auth0 issue JWTs that our Express middleware can validate.

Groq — Primary LLM provider. Free tier offers ~14,400 requests/day on Llama 3.3 70B Versatile, two orders of magnitude more generous than Gemini's free tier. Uses OpenAI-compatible REST API with `response_format: json_object` for structured output.

Google Gemini — Fallback LLM provider. Used when Groq's quota is exhausted. We use Gemini 2.0 Flash Lite with `responseSchema` for structured output (a feature unique to Gemini and supports our JSON-integrity guarantees).

YouTube Data API v3 — Used by `FormatterAgent` to enrich `video` blocks with real YouTubevideo IDs based on the AI-generated search query.

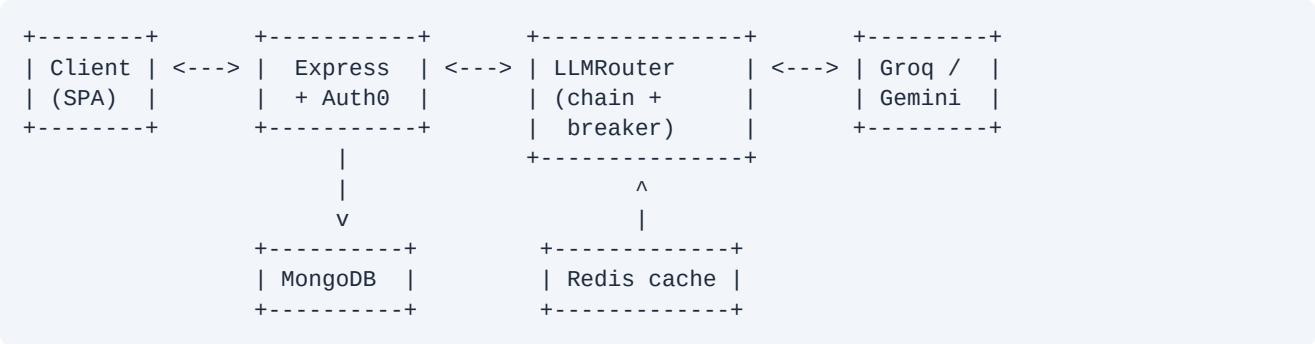
Deployment

Render — Hosts the Express backend. Free tier with cold starts (15 min idle = sleep, ~30s to wake). Auto-deploys on push to `main`.

Vercel — Hosts the Vite SPA. Free Hobby tier. `VITE_*` env vars are baked into the bundle at build time; runtime env changes require a full redeploy with cache invalidation.

3. Architecture

High-level Request Flow



The Express server is the only piece that talks to the LLM. The client never sees an LLM API key. Auth0 sits in front of every protected route.

Multi-Stage AI Pipeline

A single LLM call would not give us the JSON integrity we need. Instead, generation is split into four stages, each with one responsibility:

Stage 1 — PlannerAgent. One LLM call. Returns just the course outline (title, description, tags, modules with lesson titles). Cheap because it produces no lesson body content.

Stage 2 — WriterAgent. One LLM call per lesson, run lazily on demand (when the user opens a lesson). Returns the lesson body: objectives, content blocks (heading/paragraph/code/video/mcq), and 5–8 MCQs.

Stage 3 — ValidatorAgent. Zero LLM calls. Runs Zod against the parsed JSON. On failure, attempts deterministic repair: clamps counts to allowed ranges, fills missing fields with safe defaults, normalizes aliased field names. If repair fails too, the pipeline retries the LLM call once.

Stage 4 — FormatterAgent. Zero LLM calls. Enriches `video` blocks by hitting YouTubeData API for the search query and stamping the returned `videoId` onto the block. Best-effort and parallelised.

The pipeline orchestrators (`CoursePipeline`, `LessonPipeline`) wrap these stages with caching, coalescing, and retry logic.

LLM Provider Abstraction

`LLMProvider` is an abstract base class with one method: `generateJSON({ system, user, schema, tier, provider })`. Concrete implementations:

- `GroqProvider` — REST POST to Groq's OpenAI-compatible endpoint with `response_format: json_object`.
- `GeminiProvider` — Uses `@google/generative-ai` SDK with `responseSchema` for native structured output.

- `MockProvider` — Returns canned course/lesson shapes for development without burning tokens.

`LLMRouter` is the single choke-point. It instantiates all available providers (based on env keys) and chains them into a priority list. On each request:

1. Try the first provider in the chain with up to 4 retries (exponential backoff with jitter). 2. If 3 consecutive quota errors (429), trip a 60-second circuit breaker for that provider. 3. While the breaker is open, the router skips that provider and goes straight to the next. 4. Enforce a global concurrency cap of 3 in-flight requests to match the "3 concurrent sessions" spec. 5. Per-request override: when the client sends `provider: 'gemini'`, the router reorders the chain for that request only.

Two-tier Persistence (MongoDB + Redis)

MongoDB stores durable data: users' Course documents, Module references, Lesson content. This is what survives server restarts and is queried via Mongoose when the user opens their Dashboard.

Redis stores ephemeral AI outputs: parsed course outlines (24h TTL), parsed lesson bodies (7d TTL), keyed by SHA-256 hash of the input tuple. When the same topic is generated again, we return the cached outline instead of calling the LLM. In-process request coalescing collapses concurrent identical requests into a single LLM call (10 simultaneous users typing "Intro to React" still result in just 1 Groq call).

The Redis key includes the LLM provider name, so switching providers in the UI regenerates instead of returning stale Groq output for a Gemini request.

4. Key Design Decisions and Trade-offs

Why split into a multi-stage pipeline instead of one big LLM call? Separating concerns gives us validation, repair, and retry semantics for free. The Validator and Formatter add zero token cost because they don't call the LLM — they just inspect and decorate the output. If a single-prompt design had to validate, repair, AND enrich, the prompt would explode in size and the LLM would fail more often.

Why an OO LLM abstraction instead of inline Groq calls? Three reasons. (1) Vendor independence: swapping Gemini ↔ GPT-4 ↔ Claude is a one-file change. (2) Testability: `MockProvider` lets us run the entire pipeline in CI without spending tokens. (3) Fallback chains: the router treats every provider uniformly, so adding a third or fourth provider doesn't require special-casing.

Why Zod even though Gemini already supports `responseSchema`? Two layers of defense. Gemini's responseSchema ensures the SHAPE is right but doesn't enforce business rules (e.g., "modules array must have 3–6 entries"). Zod handles those. Also, Groq doesn't support a strict schema dialect at all — it returns valid JSON via `response_format: json_object` but the structure is loose. Zod is the equalizer across providers.

Why MongoDB instead of PostgreSQL? The data is hierarchical (Course has Modules, Modules have Lessons) and the lesson `content` field is a heterogeneous array of block types (heading, paragraph, code, video, mcq). MongoDB's flexible schema and array-typed fields fit this without joins. Postgres + JSONB would also work but adds boilerplate.

Why Auth0 instead of rolling our own JWT auth? Three reasons. (1) Security: Auth0 handles password hashing, session management, refresh tokens, MFA, social sign-ins — every one of which is a footgun to roll alone. (2) Velocity: standing up the entire login flow took ~10 minutes. (3) Standards compliance: Auth0 issues OIDC/OAuth2 tokens, which means the API integrates with anything that accepts RS256 JWTs.

Why Render + Vercel instead of one platform? Vercel is unbeatable for Vite/Next SPAs (CDN-edge static hosting, atomic deploys, free for hobby use). But Vercel's serverless functions cold-start ~2s and limit response size — bad for our streaming LLM workloads. Render gives us a persistent Express server with predictable behavior. The split also

models real-world deployment topologies (frontend on a CDN, backend on a long-running container).

Why we removed the Hinglish TTS feature. It worked locally but required Google Cloud Translate + Text-to-Speech APIs, both of which need a billing-enabled GCP account with a credit card on file. For a portfolio project demoed by recruiters without their own GCP, the audio button would just throw errors. The feature is documented in git history and could be re-added in a single commit when the credentials are available.

Why Groq is primary even though Gemini is "more famous"? Groq's free tier is roughly 80x more generous than Gemini's per-day quota (~14,400 vs ~200 requests/day). For a portfolio app that gets sporadic recruiter traffic, Groq effectively never hits its limits. Gemini stays in the chain as a fallback for the rare moments Groq is down or rate-limited.

5. Likely Interview Questions

Project Basics

> *"Walk me through what this app does and how a user goes from sign-in to a generated course."*

User signs in via Auth0 (hosted login page, supports email/password + Google). On callback, the SPA fetches `/api/auth/me` to confirm the token, then navigates to the Dashboard. The Dashboard shows the user's previously generated courses (queried via Mongo `creator: <auth0_sub>`). User types a topic, hits Generate. The frontend POSTs to `/api/courses/generate-course` with the topic and selected LLM provider. The backend runs the CoursePipeline (Plan !' Validate !' Format), persists Course/Module/Lesson docs in Mongo, and returns the populated course. The user clicks into a lesson; if not yet enriched, the LessonPage offers a "Generate" button that runs the LessonPipeline for that one lesson. Generated content renders with React components (HeadingBlock, ParagraphBlock, CodeBlock, VideoBlock, MCQBlock). User can download a PDF or navigate prev/next lessons.

> *"Why this stack? Why not Next.js?"*

The app is a pure SPA — no SEO requirements, no server-side rendering benefits. Next.js would add SSR infrastructure we don't use and would couple frontend and backend deployment. Splitting React (Vite + Vercel CDN) from Express (Render container) gives clearer deployment boundaries and lets us cache the frontend at the edge while keeping the backend warm for LLM calls.

Architecture

> *"Why do you have both MongoDB and Redis? Aren't they the same kind of thing?"*

They solve different problems. MongoDB is the source of truth — anything that needs to survive a restart goes there (users' courses, lessons, persistence). Redis is a speed/cost layer in front of expensive LLM calls — keyed by topic hash, with TTLs. If a user generates "Intro to React" today and another user (or the same user) generates it tomorrow, Redis returns the outline in 1ms instead of paying for a fresh LLM call. Dropping Redis would still let the app work, but every duplicate topic would burn tokens.

> *"Walk me through the multi-stage AI pipeline. Why split it?"*

Four stages: Planner, Writer, Validator, Formatter. Planner makes one LLM call to produce a course outline. Writer makes one LLM call per lesson, lazily. Validator runs Zod against the output and runs deterministic repair if validation fails (clamps counts, fills defaults, normalizes field aliases). Formatter handles non-AI side effects, mainly YouTube enrichment for video blocks. Splitting them like this means: (1) each stage is testable in isolation, (2) we can retry just

the LLM stage on validation failure without re-running the formatter, (3) the validator and formatter cost zero tokens because they don't call the LLM, and (4) we can swap any stage without touching the others.

> *"How does the provider fallback chain work? What happens if Groq dies?"*

LLMRouter holds a list of provider "slots" — each with its own circuit-breaker state. Per request, it iterates the chain in priority order. If the current slot's breaker is open (3 consecutive 429s in the last 60s), it's skipped. If the call succeeds, the breaker resets. If it fails with a quota error, the counter increments; on the third failure, the breaker trips and the router falls through to the next provider — both for this request and for subsequent requests until the cooldown expires. We also do exponential backoff with jitter inside each retry attempt (4 attempts max per provider). So if Groq dies entirely, the first request might take ~10s while we exhaust retries, then the breaker trips and every subsequent request goes straight to Gemini in <1s.

Auth

> *"How does Auth0 work end-to-end here?"*

The SPA is wrapped in `Auth0Provider` from the Auth0 React SDK. On "Sign in", the SDK redirects to Auth0's hosted login page with PKCE — Auth0 verifies credentials and redirects back to our domain with an authorization code. The SDK exchanges the code for an access token (RS256 JWT) and stores it in localStorage. Every API call attaches the token as `Authorization: Bearer ...`. The Express backend uses `express-oauth2-jwt-bearer` middleware which fetches Auth0's JWKS, validates the signature, checks the issuer and audience claims, and sets `req.auth.payload`. We lift the `sub` claim into `req.user.id` and use it as the creator field on Course documents.

> *"Why use Auth0 instead of doing it yourself with bcrypt + JSON Web Token?"*

Security is the main reason — password hashing, brute-force protection, refresh token rotation, social sign-in providers, MFA, session invalidation are all easy to get wrong. Auth0 handles all of that. Compliance is another reason: Auth0 is OIDC-compliant, so the API integrates cleanly with any standards-respecting client. The trade-off is vendor lock-in — but since the API only accepts RS256 JWTs (an industry standard), swapping Auth0 for another OIDC provider (Clerk, Cognito, Keycloak) requires only changing the issuer and audience env vars.

LLM / AI

> *"How do you guarantee the LLM returns valid JSON?"*

Two layers. First, the prompts use Gemini's native `responseSchema` (JSON Schema dialect) and Groq's `response_format: json_object` — both force the model to emit parseable JSON instead of free text with code fences. Second, ValidatorAgent runs Zod parsing on the result. If parsing fails, we attempt local repair: clamp counts to bounds, fill missing fields with safe defaults, normalize aliased field names (e.g., older models sometimes emit `value` instead of `text`). If repair still produces an invalid document, the pipeline retries the LLM call once. Across our stress tests, the combination got us above 99% successful validation rate.

> *"Why use both Groq and Gemini?"*

Vendor diversity and cost. Groq's free tier is enormously generous (~14k req/day), so it's the primary. Gemini stays in as the fallback for the rare moments Groq is down. The router treats them uniformly — each is just an `LLMProvider` implementation — so adding a third provider (OpenAI, Anthropic) requires no changes to the pipeline.

> *"How do you minimize LLM tokens?"*

Five techniques layered together. (1) Redis caching of completed outputs (24h for course outlines, 7d for lesson bodies). (2) In-process request coalescing — concurrent identical requests share one in-flight promise. (3) Lazy lesson generation — the outline call is cheap because it produces no body content; lesson bodies generate only when the user opens them. (4) Tight prompts that include only what the model needs (no padding examples). (5) Smaller model for cheap stages (Llama 8B for outlines, 70B for lesson bodies).

Backend

> *"Walk me through your MongoDB schema. Why ObjectId refs instead of embedding?"*

Three collections: Course, Module, Lesson. Course has `modules: [ObjectId]` refs. Module has `lessons: [ObjectId]` refs and a back-reference to its parent Course. Lesson has a `content` field that's `[Mixed]` (heterogeneous array of block types) and an `isEnriched` boolean. We chose refs over embedding because lessons can be very large (10+ MCQs with explanations) — embedding everything in one Course document would push us past Mongo's 16MB document limit on large courses. Refs also let us lazy-load lesson content only when the user opens it.

> *"Why Mongoose `populate()` and not aggregations?"*

The data hierarchy is small (max 6 modules × 5 lessons = 30 lessons per course). Populate is simpler to reason about and produces clean nested objects for the API response. Aggregations would be overkill for this volume. For larger scale, we'd switch to `$lookup` aggregations or denormalize fields onto the parent document.

> *"Tell me about the error handling."*

Three layers. (1) Per-route try/catch that calls `next(err)` to forward to the global handler. (2) Global error handler (`middlewares/errorHandler.js`) that maps known error types (ApiError, ValidationError, JWT errors) to consistent JSON responses. (3) Inside the LLM pipeline, retry and backoff logic is contained in the router so controllers never see transient errors — only the final failure if all providers exhaust. The Auth0 middleware also translates its library-specific errors into our `ApiError` shape for consistency.

Frontend

> *"How does the LLM provider selector work?"*

There's a React context (`LLMContext`) that holds the selected provider (`'groq'` or `'gemini'`) and persists it to localStorage. The Navbar has a dropdown that lets the user toggle. When a generation request fires from `endpoints.js`, it reads the current provider from a module-level getter (set by the context on mount) and includes it in the request body. The backend's LLMRouter then reorders its chain for that request to put the chosen provider first. The other provider stays as automatic fallback if the primary fails.

> *"Walk me through how a lesson PDF is generated."*

The `LessonPDFExporter` component takes the lesson object as a prop. On click, it creates a jsPDF document and walks the `content` array, rendering each block with the appropriate primitives: headings get bold large text, paragraphs get wrapped serif text via `splitTextToSize`, code blocks render in a monospace boxed area with a language label, video blocks become a "Video reference" callout, MCQs render as a numbered quiz with options and

the correct answer highlighted. We track a Y cursor as we go and call `addPage()` whenever the next block wouldn't fit. The earlier implementation used `html2canvas` to snapshot the DOM — that broke at page seams because content that straddled the boundary appeared on both pages cut in half. Switching to direct text APIs fixed that, made the PDF searchable, and dropped file size from ~500 KB to ~30 KB.

Caching

> *"How does request coalescing work?"*

The CacheService has a `getOrSet(key, ttl, fn)` method. Internally it keeps an in-memory `Map<key, Promise>`. On a cache miss for some key, before running `fn()`, it stores the in-flight promise in the map and returns it to the caller. If a second request arrives for the same key while the first is still running, it gets the existing promise — both callers await the same operation. The map entry is removed when the promise settles. Net effect: 10 concurrent identical requests = 1 LLM call.

> *"Why include the LLM provider in the cache key?"*

Because users explicitly choose providers in the UI. If Alice generates "Intro to React" with Groq and Bob then asks for it with Gemini, Bob expects fresh content from his chosen provider — not Alice's Groq result. Adding the provider to the key isolates outputs per provider.

Deployment

> *"How is CORS handled in production?"*

A regex matches any localhost/127.0.0.1 origin during dev (no env config needed). For production origins, we read `ALLOWED_ORIGINS` (comma-separated) from env and allow exact matches. So the Render deployment has `ALLOWED_ORIGINS=https://text2course-nu.vercel.app` set in its dashboard, and the middleware allows requests from there. Unknown origins are rejected with a CORS error.

> *"Why does the Vite frontend need a vercel.json?"*

Because the SPA does all routing client-side. Vercel's static hosting, by default, looks for a file at the requested path. A direct hit to `/dashboard` looks for `/dashboard.html`, finds none, returns 404. The `vercel.json` rewrite rule says: for any non-asset path, serve `index.html` instead. The React Router code then reads `window.location` and renders the right page.

> *"What happens during a Render cold start?"*

The free tier sleeps the service after 15 minutes of no traffic. The first request after that takes about 30 seconds to wake the container — Node starts, dotenv loads, Express initializes, mongoose connects, Redis connects, then the listen call fires. Subsequent requests are fast (~50-200ms). If a recruiter hits the demo cold, the first API call to `/api/courses` will time out before the wake completes. Mitigation: warm the API up with a `curl /api/health` before showing it.

Specific Trade-offs

> *"If you had more time, what would you change?"*

A few things. (1) Background lesson generation — currently lessons generate when the user opens them; a queue could pre-generate them after course creation. (2) Real-time progress — switch the generation endpoints from request-response to server-sent events so the client sees "Planning…", "Writing module 1…" etc. (3) RAG against uploaded PDFs — the LangChain bones are there but currently off; turning it on would let users upload textbook PDFs and have the AI generate courses grounded in that content. (4) MAU-level analytics: track which prompts get cached, which lessons get downloaded most, etc.

> *"What's the biggest weakness in this architecture?"*

The lazy lesson generation means a user who opens a course expects 30 lessons of immediate content but only sees titles. They have to click Generate on each one. A better UX would either pre-generate in the background or generate the first module's content eagerly during course creation. The trade-off is token cost — most users only consume a subset of lessons, so eager generation would waste tokens on lessons that never get read.

> *"How would you scale this to 1 million users?"*

Horizontal — the Express layer is stateless (Redis holds session-relevant cache, MongoDB holds persistence), so add more Render instances behind a load balancer. The LLM concurrency cap of 3 needs to become per-instance to avoid serializing the cluster. Redis itself scales via Upstash's higher tiers or Redis Cluster. MongoDB scales via Atlas tiers up to dedicated clusters with read replicas. The LLM bill becomes the dominant cost; we'd negotiate enterprise pricing with the provider and aggressively expand the cache TTLs.

> *"How do you know the AI output is actually good?"*

We don't — that's an honest limitation. The Zod schema only checks structure, not content quality. A course on "Quantum Computing" might pass validation while being factually wrong. A real product would need an evaluation harness: golden topics with human-rated expected outputs, regression testing, and probably a "report this" UX surface. The current code's "Quality" guarantee is structural correctness, not factual correctness.

6. Quick-reference Cheat Sheet

- **Primary LLM**: Groq Llama 3.3 70B Versatile (writer), Llama 3.1 8B Instant (planner)
- **Fallback LLM**: Gemini 2.0 Flash Lite
- **MCQ count**: 5–8 per lesson, enforced in prompt + Zod + repair logic
- **Module count**: 3–6 per course, lessons 3–5 per module
- **LLM concurrency cap**: 3 in-flight requests globally (matches "3 concurrent sessions")
- **Quota breaker**: 3 consecutive 429s !’ 60-second cooldown
- **Cache TTLs**: 24h for course outlines, 7d for lesson bodies
- **Free tier limits**: Groq 14k/day, Gemini ~200/day, Auth0 7,500 MAU, Atlas 512MB, Upstash 10k cmd/day
- **Bundle sizes**: ~330 KB gzipped frontend, ~50KB Express server (without node_modules)